

【第7回】 **jQuery**とモダンフレームワークについて

1. jQueryについて

1-1. jQueryとは？

jQueryとは、**JavaScriptを短く、簡単に、ブラウザに依存せずに記述するためのライブラリ**のこと。
ブラウザごとに実装したJavaScriptの挙動が違ったり、イベントの設定の記述が手間だったりすることから、全ブラウザ共通でスクリプトの実装ができ、実装時の記述量を減らせるようにjQueryができた。

1-2. jQueryの使い方

jQueryは、jsファイルで作られているため、通常のJavaScriptと同様に、**HTMLのheadタグ内でjsファイルを読み込む**ことで、以下のjsファイル内で使用することが可能となる。

jQueryを使用する際には、**jQueryが使用されるスクリプトよりも前にjQueryのjsファイルを読み込む必要がある**ので、注意が必要である。

また、jQueryの読み込み方法には、2種類あり使用する読み込み方法を適切に選択する必要がある。

例)

- ・CDN(Content Delivery Network)を使用した例

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
```

→ インターネット上に分散されているサーバーの中からより近いサーバーのファイルを自動で読み込み使用する。

インターネット環境を使用するため、オンラインの場合のみ使用可能。

- ・ファイルをダウンロードして使用した例

```
<script src="js/jquery-3.7.1.min.js"></script>
```

→ WEBからjQueryのjsファイルをダウンロードし、該当のHTMLファイルから参照可能な場所に配置し、読み込む。

一度ダウンロードしてしまえば、**配置を変えない限り使用可能なため、オフラインでの使用可能。**

1. jQueryについて

1-3. jQueryでの記述

jQueryでは、一般的にJavaScriptで可能なDOM操作を簡略化して記述することが可能である。

- ・要素の取得

`$([セレクトア])`

`$('#btn')` → `document.getElementById('btn')`と同じ

`$('.cls')` → `document.getElementsByClassName('cls')`と同じ

`$('div')` → `document.getElementsByTagName('div')`と同じ

- ・DOM操作

`$([セレクトア]).text();` → 指定した要素内の文字を取得する

`$([セレクトア]).text([文字列]);` → 指定した要素内の文字を設定・変更する

→ `.textContent`と同じ動作をする

`$([セレクトア]).html();` → 指定した要素内のhtmlタグを取得する

`$([セレクトア]).html([タグ]);` → 指定した要素内のhtmlタグを設定・変更する

→ `.innerHTML`と同じ動作をする

`$([セレクトア]).val();` → 指定した要素のvalue属性を取得する

`$([セレクトア]).val([値]);` → 指定した要素のvalue属性を設定・変更する

→ `.value`と同じ動作をする

`$([セレクトア]).css();` → 指定した要素のstyle属性を取得する

`$([セレクトア]).css('スタイル名', 'スタイルの値');` → 指定した要素のstyle属性を追加する

→ `.style`と同じ動作をする

1. jQueryについて

1-3. jQueryでの記述

- DOM操作

`$([セレクトア]).attr('属性名');` → 指定した要素の指定の属性を取得する

`$([セレクトア]).attr('属性名', '属性の値');` → 指定した要素の指定の属性を設定・変更する

→ `.getAttribute`・`setAttribute`と同じ動作をする

- イベント処理

`$([セレクトア]).on('イベント名', [イベントで処理する内容] or [イベントで呼び出す関数]);`

→ 指定した要素に指定した動作のイベントを付与する

→ `addEventListener`と同じ動作をする

- `this`の扱い

`$(this)` → 使用するイベントに関連づいた要素を取得する

→ `this`と同じ動作をする

- Ajax

`$.ajax({
 url:[APIのURL]
 ,method:["GET" or "POST" or "PUT" or "DELETE"]
 ,success:[API処理成功時のコールバック関数]
 ,error:[API処理エラー時のコールバック関数]
});` → APIを利用した処理

→ `fetch`と同じ動作をする

1. jQueryについて

1-4. 現在のjQueryとJavaScript

jQueryは、前に説明した通り、**JavaScript**をよりコーディング量を少なくして、簡単に実装するのに非常に便利であるが、現在では「役目を終えた技術」と言われており、**JavaScript**のモダンフレームワーク(**React JS**や**Angular JS**)とは相性が悪いと言われている。

とはいえ、既存の**WEB**ページやプラグインなどでは、**jQuery**を使用しているものがあるため、**jQuery**を理解しておく必要がある。

【例題1】

Javascript-sample-q1.htmlの要素に対して、**jQuery**を使って、以下の操作を行う**JavaScript**を実装しなさい。

- ① **HTML**ファイルの読み込み完了時に、現在のタイトルの後ろに「の勉強」の文字を追加して表示する。
- ② **HTML**ファイルの読み込み完了時に、**p**タグの文字を取得して、ダイアログに表示する。
- ③ **HTML**ファイルの読み込み完了時に、**span**タグの文字色を赤色に変更する。
- ④ 入力項目1・入力項目2の背景色をグレー(**#F3F3F3**)に変更する。
- ⑤ 入力項目2に「入力項目2」というプレースホルダ属性(**placeholder**)を追加する。

【例題2】

Javascript-sample-q2.htmlの要素に**jQuery**を使用してイベントを実装し、以下のような処理を行えるようにしなさい。

- ① **HTML**ファイルの読み込み完了時に、「**HTML**の読み込みが完了しました」とダイアログに表示する。
- ② 入力項目1からカーソルが離れた際に、「入力項目1からフォーカスが離れました」とダイアログに表示する。
- ③ 入力項目2に「**a**」を入力した場合にのみ、「**[a]**が入力されました」とダイアログに表示する。
- ④ 入力項目5が選択された場合に、「入力項目5が選択されました」とダイアログに表示する。
- ⑤ ボタンをクリックされた場合に、「ボタンがクリックされました」とダイアログに表示する。

2. モダンフレームワークについて

2-1. JavaScriptのモダンフレームワークとは?

JavaScriptのモダンフレームワークとは、大規模・長期運用のWEBアプリやWEBページを破綻せずに作るために生まれた仕組みのこと。

モダンフレームワークでは、これまでのjQueryとは異なる思想でのWEBページ構築が実現可能である。

2-2. jQueryとモダンフレームワークの思想の違い

前に説明した通り、これまでのjQueryとモダンフレームワークとはWEBページを構築する上での思想が異なる。jQueryでは、JavaScriptでDOMを直接操作し、見た目や値を変更することを前提としているため、「DOM操作が全て」であったが、モダンフレームワークでは実装しているスクリプトの中で画面の状態に応じて表示を切り替えるコンポーネント指向という考え方で開発される。

jQueryにしても、モダンフレームワークにしても、ベースとなるJavaScriptの記述に関しては、これまでの学習内容が基本となる。

2-3. 代表的なモダンフレームワーク

名前	説明	特徴
React	WEBアプリケーションのUIを効率的に構築するためのJavaScriptライブラリ	柔軟で事実上の標準
Vue	WEBページのUIを構築するためのJavaScriptフレームワーク	勉強しやすい
Angular	Googleが開発したTypeScriptベースのWEBアプリケーションフレームワーク	機能が充実していて堅牢

2. モダンフレームワークについて

2-3. 代表的なモダンフレームワーク

名前	説明	特徴
Svelte	コードを高速・軽量なJavaScriptへ変換する次世代のWEBアプリケーションフレームワーク	実行時が軽い

WEBページ・**WEB**アプリケーションのみならず、モバイルアプリが主流となる今、開発の現場でも上記のようなモダンフレームワークを利用した開発を取り入れていくことが必要不可欠となる。